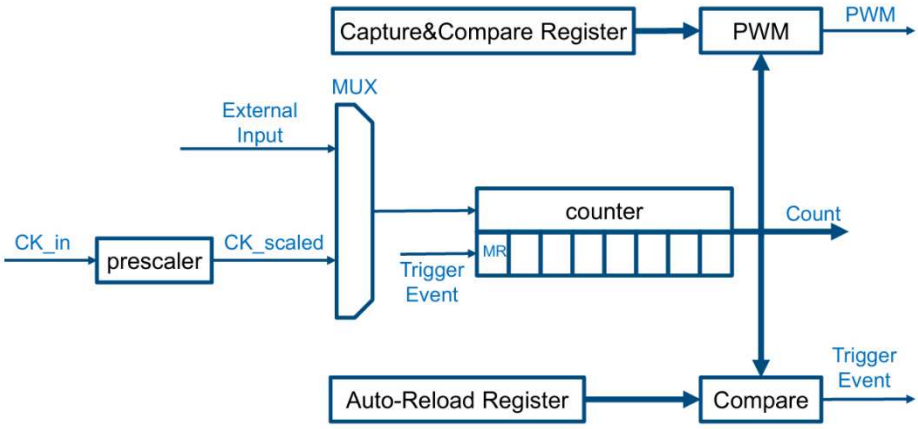
03) Timers

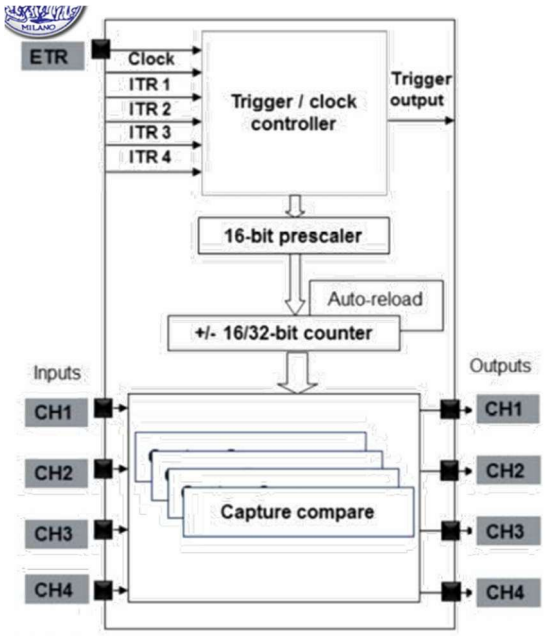
Il CK_in è un clock, e il **prescaler** serve a modularlo. Poi questo segnale CK_scaled va dentro un **Multiplexer** perché potremmo voler contare i clock oppure degli eventi esterni. Poi c'è il **counter** che conta da 0 oppure da un numero fino a zero. Noi possiamo leggere questo counter, oppure utilizzare questa "conta" in due modi:



- **Auto-Reload Register**: compara il valore del contatore con uno che settiamo noi e triggera un evento quando il contatore raggiunge quel numero.
- **PWM (Pulse Width Modulator)**: generato tramite il Capture & Compare Register.

Blocchi interni di un Timer

1. Trigger / Clock Controller



È il blocco che **gestisce il clock e i segnali di trigger del timer**.

- **Selects input clock** → sceglie da quale sorgente arriva il clock del timer.
- **Synchronization with other clocks** → permette di sincronizzare più timer tra loro.

Ad esempio:
Timer Master → Output Trigger (TRGO) → Timer Slave → Input Trigger (TRGI)
In questo modo un timer può comandare o sincronizzarsi con un altro timer.

2. Prescaler

Il **prescaler** serve a ridurre la frequenza con cui il contatore viene incrementato. Se metto PSC=0, conto ogni clock, se metto PSC=1, conto un clock sì ed uno no, se metto PSC=3 conto un clock e ne skippo 2 e così via…
Se il timer riceve un clock di **72 MHz** e imposto PSC=71 il contatore viene incrementato a:

$$f_{Timer} = \frac{f_{Clock}}{PSC + 1} = \frac{72\ MHz}{71 + 1} = 1\ MHz$$

Allo stesso modo:

$$\Delta T = \frac{PSC + 1}{f_{Clock}} = \frac{71 + 1}{72\ MHz} = 1\ \mu s$$

quindi il counter aumenta di 1 ogni **1 µs**.
Il prescaler **non fa arrivare ogni impulso di clock al counter**, ma ne fa passare uno ogni PSC+1.

3. Counter

È il **contatore vero e proprio** del timer. Può funzionare in diverse modalità:

Modalità	Funzionamento
Up	0 → 1 → 2 → 3 → ...
Down	... → 3 → 2 → 1 → 0

4. Capture/Compare Registers

Sono registri associati ai **canali del timer**. **Ogni canale ha il proprio registro**:

- **Channel 1 → CCR1**
- **Channel 2 → CCR2**
- **Channel 3 → CCR3**
- **Channel 4 → CCR4**

Possono essere utilizzati principalmente in due modi:

- **Input Capture** → memorizzare il valore del counter quando arriva un determinato evento sul pin (es. quando premo o rilascio un bottone).
- **Output Compare** → confrontare continuamente il counter con un valore nel registro e generare un evento quando coincidono.

Ad esempio: quando CNT = CCR1 → viene generato un evento di **compare** sul Channel 1.

5. Auto-Reload Register (ARR)

L'**ARR** stabilisce il valore massimo fino al quale conta il timer, quindi determina il suo **periodo**. Setta il FSR del Counter. In modalità Up: quando il counter raggiunge ARR, il timer può **ripartire da 0**.
Per questo l'ARR è fondamentale per impostare la frequenza con cui si verificano gli eventi periodici del timer.
La frequenza del timer può essere espressa, semplificando, come:

$$f_{Timer} = \frac{f_{Clock}}{(PSC + 1)(ARR + 1)}$$

Quindi:

- **Prescaler (PSC)** → decide **quanto velocemente** conta il counter.
- **ARR** → decide **fino a dove** conta.
- **Counter (CNT)** → contiene il **valore corrente del conteggio**.
- **CCR** → permette di fare **capture/compare**.
- **Trigger/Clock Controller** → decide **da dove arriva il clock e come sincronizzare il timer**.

In sintesi, puoi immaginare il timer così:

Clock → Prescaler → Counter → confronto con ARR / CCR → evento.

PWM

$$f_{PWM} = \frac{f_{Timer}}{(ARR + 1)(PSC + 1)}$$

$$DC = \frac{CCR_x + 1}{ARR + 1}$$

Timer Functions

1. HAL_TIM_PWM_Start()

HAL_StatusTypeDef HAL_TIM_PWM_Start(TIM_HandleTypeDef *htim, uint32_t Channel)

Avvia la generazione del segnale PWM sul canale specificato.

- htim → indica quale timer utilizzare, ad esempio &htim1.
- Channel → indica il canale, ad esempio TIM_CHANNEL_1.

2. HAL_TIM_PWM_Stop()

HAL_StatusTypeDef HAL_TIM_PWM_Stop(TIM_HandleTypeDef *htim, uint32_t Channel)

Fa l'opposto della precedente: **ferma la generazione del PWM** sul canale indicato.

Timer Base

Le funzioni HAL_TIM_Base_* servono invece a utilizzare il timer come **base temporale**, cioè facendo semplicemente contare il timer.

3. HAL_TIM_Base_Start()

HAL_StatusTypeDef HAL_TIM_Base_Start(TIM_HandleTypeDef *htim)

Avvia il timer, senza generare interrupt. La CPU non viene automaticamente avvisata quando il counter raggiunge ARR.

4. HAL_TIM_Base_Start_IT()

HAL_StatusTypeDef HAL_TIM_Base_Start_IT(TIM_HandleTypeDef *htim)

Avvia il timer **abilitando anche gli interrupt**. Quando il counter raggiunge il valore di ARR, si verifica un **Update Event**, che può generare un interrupt. Questa è una funzione molto utile per eseguire periodicamente del codice.

5. HAL_TIM_Base_Stop()

HAL_StatusTypeDef HAL_TIM_Base_Stop(TIM_HandleTypeDef *htim)

Ferma il timer. Il counter smette di essere incrementato.

6. HAL_TIM_Base_Stop_IT()

HAL_StatusTypeDef HAL_TIM_Base_Stop_IT(TIM_HandleTypeDef *htim)

Ferma il timer **e disabilita la generazione degli interrupt** associati al timer. Quindi:

- Base_Stop() → ferma il timer.
- Base_Stop_IT() → ferma il timer **e disabilita gli interrupt**.

7. HAL_TIM_PeriodElapsedCallback()

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)

Questa è una **callback** che viene chiamata quando il timer raggiunge la fine del suo periodo, cioè quando avviene l'**Update Event**. Se, ad esempio, TIM2 genera un interrupt ogni **1 ms**, questa funzione viene eseguita ogni **1 ms**.

Funzione	**Cosa fa**
HAL_TIM_PWM_Start()	Avvia il **PWM** su un canale
HAL_TIM_PWM_Stop()	Ferma il **PWM** su un canale
HAL_TIM_Base_Start()	Avvia il timer **senza interrupt**
HAL_TIM_Base_Start_IT()	Avvia il timer **con interrupt**
HAL_TIM_Base_Stop()	Ferma il timer
HAL_TIM_Base_Stop_IT()	Ferma il timer e gli interrupt
HAL_TIM_PeriodElapsedCallback()	Codice eseguito quando il timer raggiunge **ARR** e genera l'interrupt

Progetti

Lab03_1c

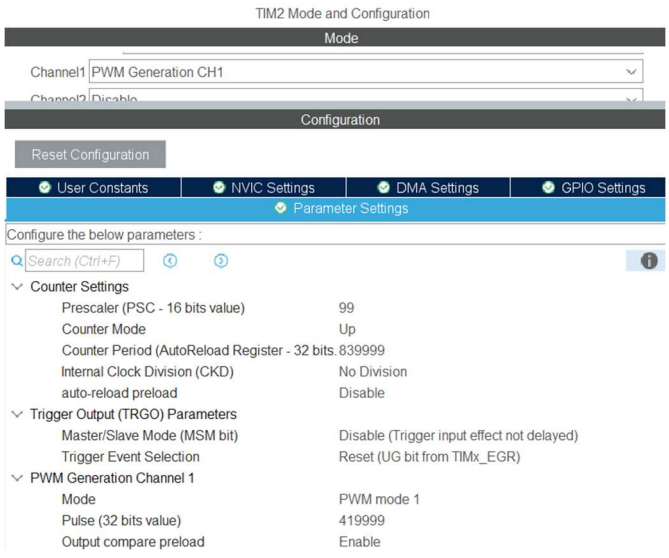
Lab03_1c: fai blinkare LD2 a 1Hz con DC=50% usando il PWM

Pin Configuration (ioc)

- LD2 (Green Led):
  - PA5
  - TIM2_CH1

Timer

- Prescaler = 99
- ARR = 839999
- Pulse = 419999



$$f_{PWM} = \frac{f_{Timer}}{(ARR+1)(PSC+1)} = \frac{84\ MHz}{(839999+1)\cdot(99+1)} = 1Hz$$

$$DC = \frac{CCR_x + 1}{ARR + 1} = \frac{419999 + 1}{839999 + 1} = 50\%$$

Lab03_2a

Lab03_2a: Usa il PWM per suonare il La4 (440Hz, DC=50%) per 3 secondi.

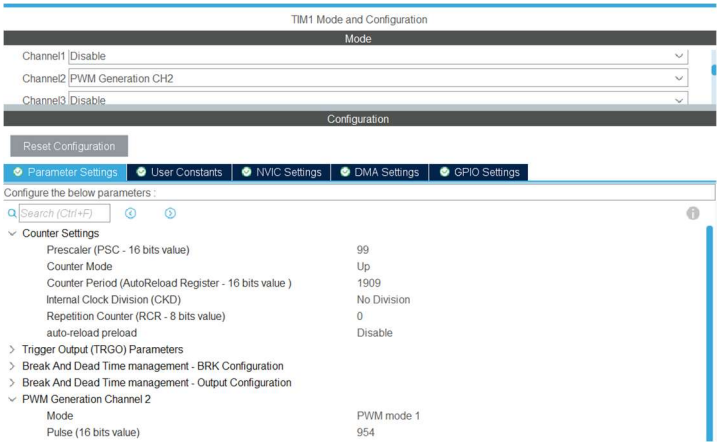
File ioc

Pin

- Speaker: PA9 = TIM1_CH2

Timers

- TIM1 (Speaker):
  - Channel 2: PWM GENERATION CH2
  - Prescaler: 99
  - ARR: 1909
  - Pulse: 954



$$f_{PWM} = \frac{f_{Timer}}{(ARR+1)(PSC+1)} = \frac{84\ MHz}{(1909+1)\cdot(99+1)} = 440Hz$$

$$DC = \frac{CCR_x + 1}{ARR+1} = \frac{954+1}{1909+1} = 50\%$$

- TIM2 (3 Seconds):
  - Tim2 global interrupt: Enable
  - Clock Source: Internal Clock
  - Prescaler: 99
  - ARR: 839999

$$f_{PWM} = \frac{f_{Timer}}{(ARR+1)(PSC+1)} = \frac{84\ MHz}{(839999+1)\cdot(99+1)} = 1Hz$$

Codice

Private Variables

```
int num_secondi=0;
```

User Code 0 (prima del main)

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
	if(htim->Instance == TIM2){
		num_secondi+=1;
		if(num_secondi==3){
			HAL_TIM_PWM_Stop(&htim1, TIM_CHANNEL_2);
			HAL_TIM_Base_Stop_IT(&htim2);
		}
	}
}
```

User Code 2 (dentro il main, prima del while)

```
// Iniziamo a suonare
```

```
HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_2);
// Iniziamo a contare 3 secondi
HAL_TIM_Base_Start_IT(&htim2);
```

Lab03_2b

Lab03_2b: suona una canzone con lo Speaker.

File .ioc

Pin

- Speaker: PA9: TIM1_CH2

Timers

- Timer 1 (imponiamo LA4 all'inizio)
  - Clock Source: Internal Clock
  - Channel 2: PWM Generation CH2
  - PSC: 99
  - ARR: 1909
  - Pulse: 954

Per le altre note, imponendo PSC=99:

Nota	Frequency (PWM)	Period (ARR)	CCR (Pulse)
DO4	262	3206	1603
DO#4	277	3032	1516
RE4	294	2857	1429
RE#4	311	2701	1350
MI4	330	2545	1273
FA4	349	2407	1203
FA#4	370	2270	1135
SOL4	392	2143	1071
SOL#4	415	2024	1012
LA4	440	1909	954
LA#4	466	1803	901
SI4	494	1700	850

- Timer 2 (Metronomo):
  - Tim2 global interrupt: Enable
  - Clock Source: Internal Clock
  - Prescaler: 99
  - ARR: 209999

$$T_{Timer} = \frac{(ARR + 1)(PSC + 1)}{f_{Clock}} = \frac{(419999 + 1)(99 + 1)}{84\ MHz} = 500ms$$

Codice

Private TypeDef

```
// Definiamo una struttura "Note"
typedef struct {
    int Period;
    int Duration;
} Note;
```

Private Define

```
#define PeriodD0 3205
#define PeriodRE 2856
#define PeriodMI 2544
#define PeriodFA 2405
#define PeriodSOL 2141
#define PeriodLA 1908
#define PeriodSI 1699
```

Private Variables

```
int i=0;
int ticks=0;
uint32_t duration_note=500; // valore a caso
Note Song[] = {
    {PeriodSOL, 3},
    {PeriodLA, 1},
    {PeriodSOL, 2},
    {PeriodFA, 2},
    {PeriodMI, 2},
    {PeriodFA, 2},
    {PeriodSOL, 3}
};
// DICHIARAZIONE GLOBALE PER LA CONFIGURAZIONE PWM (CRITICO!)
TIM_OC_InitTypeDef sConfigOC = {0};
```

Private Function Prototype

```
void play_note(uint32_t Period, uint32_t Duration);
```

Funzioni (Definizione)

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
	if(htim->Instance == TIM2){
		ticks+=1;
		if(ticks==duration_note){
			i+=1;
			if(i==7){ // Fine Canzone
				HAL_TIM_PWM_Stop(&htim1, TIM_CHANNEL_2);
				HAL_TIM_Base_Stop_IT(&htim2);
			}else{
				play_note(Song[i].Period, Song[i].Duration);
			}
		}
	}
}

void play_note(uint32_t Period, uint32_t Duration){
    duration_note=Duration;
    ticks=0;
```

```
      // Aggiorna il Periodo (frequenza) nel registro del Timer
    // L'unica variabile da ri-inizializzare è quella di base (più veloce di
HAL_TIM_Base_Init)
    // Usiamo una macro veloce che accede direttamente al registro
    __HAL_TIM_SET_AUTORELOAD(&htim1, Period - 1);

    // Aggiorna il Pulse (Duty Cycle) nella struct sConfigOC
    sConfigOC.OCMode = TIM_OCMODE_PWM1; // Modalità PWM
    sConfigOC.Pulse = Period / 2;       // Duty cycle al 50%
    sConfigOC.OCPolarity = TIM_OCPOLARITY_HIGH;
    sConfigOC.OCFastMode = TIM_OCFAST_DISABLE;

    // Applica la nuova configurazione del canale
    if (HAL_TIM_PWM_ConfigChannel(&htim1, &sConfigOC, TIM_CHANNEL_2) != HAL_OK)
    {
        Error_Handler();
    }

    // Fai suonare nota
      HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_2);
}
```

Main (prima del while, dopo le Inizializzazioni)

```
play_note(Song[i].Period, Song[i].Duration);
HAL_TIM_Base_Start_IT(&htim2);
```

Project Lab03_2c

Lab03_2c: suona la canzone quando il microfono acquisisce un suono forte.

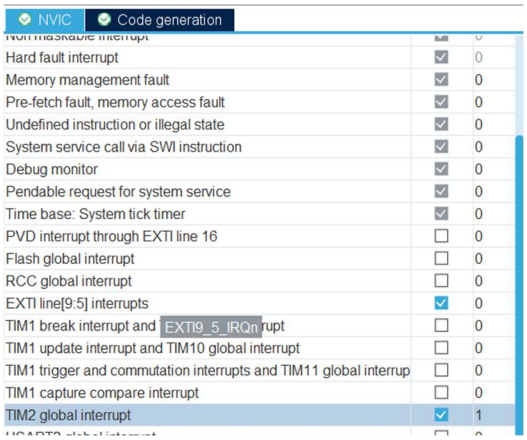
File .ioc

Pin

- Speaker: PA9: TIM1_CH2
- Microfono: PA8=GPIO_EXTI18

Timers

- Timer 1 (imponiamo LA4 all'inizio)
    - Clock Source: Internal Clock
    - Channel 2: PWM Generation CH2
    - PSC: 99
    - ARR: 1909
    - Pulse: 954
- Timer 2 (Metronomo):
    - Tim2 global interrupt: Enable
    - Clock Source: Internal Clock
    - Prescaler: 99
    - ARR: 209999



NVIC

- Microfono (secondario): EXTI line[9:5] interrupts = 0
- Timer 2 (priorità alta): TIM2 global interrupt = 0

Codice

Private TypeDef

```
// Definiamo una struttura "Note"
typedef struct {
    int Period;
    int Duration;
} Note;
```

Private Define

```
#define PeriodD0 3205
#define PeriodRE 2856
#define PeriodMI 2544
#define PeriodFA 2405
#define PeriodSOL 2141
#define PeriodLA 1908
#define PeriodSI 1699

#define PORT_MCO GPIOA
#define PIN_MCO GPIO_PIN_8

#define DEBOUNCE_MS 500
```

Private Variables

```
int i=0;
int ticks=0;
uint32_t duration_note=5; // valore a caso
uint32_t last_end_tick = 0;
volatile uint8_t playing = 0; // flag
Note Song[] = {
    {PeriodSOL, 3},
    {PeriodLA, 1},
    {PeriodSOL, 2},
    {PeriodFA, 2},
    {PeriodMI, 2},
    {PeriodFA, 2},
    {PeriodSOL, 3}
};
```

Funzioni, fuori dal Main

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if (GPIO_Pin == PIN_MCO && !playing && (HAL_GetTick() - last_end_tick > DE-
BOUNCE_MS))
    {
        // conferma il trigger con alcune letture ravvicinate, per scartare glitch di
rumore
        uint8_t confirmed = 1;
        for (int k = 0; k < 5; k++) {
            if (HAL_GPIO_ReadPin(PORT_MCO, PIN_MCO) != GPIO_PIN_SET) {
                confirmed = 0;
                break;
            }
            for (volatile int d = 0; d < 200; d++); // piccolo ritardo (qualche micro-
secondo)
        }
        if (!confirmed) return;

        playing = 1;
```

```
        i = 0;
        play_note(Song[i].Period, Song[i].Duration);
        HAL_TIM_Base_Start_IT(&htim2);
    }
}

void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
      if(htim->Instance == TIM2){
            ticks+=1;
            if(ticks==duration_note){
                  i+=1;
                  if(i==sizeof(Song)){
                        HAL_TIM_PWM_Stop(&htim1, TIM_CHANNEL_2);
                        HAL_TIM_Base_Stop_IT(&htim2);
                        playing = 0;
                        last_end_tick = HAL_GetTick();
                  }else{
                        play_note(Song[i].Period, Song[i].Duration);
                  }
            }
      }
}

void play_note(uint32_t Period, uint32_t Duration){
    duration_note=Duration;
    ticks=0;

      // Aggiorna il Periodo (frequenza) nel registro del Timer
    // L'unica variabile da ri-inizializzare è quella di base (più veloce di
HAL_TIM_Base_Init)
    // Usiamo una macro veloce che accede direttamente al registro
    __HAL_TIM_SET_AUTORELOAD(&htim1, Period - 1);

    // Aggiorna il Pulse (Duty Cycle) nella struct sConfigOC
    sConfigOC.OCMode = TIM_OCMODE_PWM1; // Modalità PWM
    sConfigOC.Pulse = Period / 2;       // Duty cycle al 50%
    sConfigOC.OCPolarity = TIM_OCPOLARITY_HIGH;
    sConfigOC.OCFastMode = TIM_OCFAST_DISABLE;

    // Applica la nuova configurazione del canale
    if (HAL_TIM_PWM_ConfigChannel(&htim1, &sConfigOC, TIM_CHANNEL_2) != HAL_OK)
    {
        Error_Handler();
    }

    // Fai suonare nota
      HAL_TIM_PWM_Start(&htim1, TIM_CHANNEL_2);
}
```